

Binary Decision Diagrams Unchained

Jaehyeok Choi 

Iowa State University, Ames, IA, USA
jaehyeok@iastate.edu

Andrew S. Miner 

Iowa State University, Ames, IA, USA
asminer@iastate.edu



Abstract—We introduce Unchained Binary Decision Diagrams (UBDDs), a new variant of Binary Decision Diagrams (BDDs) with a simple but expressive set of edge rules, including complement and swap flags. These new UBDDs are able to exploit generalized forms of common patterns that are only partially supported by existing single-rule BDDs, including BDDs and zero-suppressed BDDs, and multiple-rule BDDs, including chained BDDs, tagged BDDs, and RexBDDs. We provide reduction rules for UBDDs and show that the reduced form is canonical, and give some algorithms for UBDD manipulation. We show that UBDDs can never be worse than a constant factor larger than existing forms, and can be arbitrarily smaller for certain functions. Finally, we present experimental evidence that UBDDs can perform extremely well on practical verification applications.

I. INTRODUCTION

Binary Decision Diagrams (BDDs) and variants thereof have enabled the computer industry to tackle verification and optimization of many real-life systems [1], [2], [3], [4], [5], ensuring their correctness by finding subtle bugs in remote corner cases and allowing engineers to explore better alternatives that still satisfy the design constraints. They are a graph-based data structure for representing functions of L Boolean variables, similar to finite automata, and most BDD operations have a worst-case complexity based on the graph sizes of the operands. While the maximum size of a BDD is exponential in L , many practical functions of interest have much smaller BDD encodings.

The compactness of BDDs, at least for the variants that we consider, is due to two factors. The first is *canonicity*: for a fixed variable ordering, functions are equivalent if and only if they are represented by isomorphic graphs; this allows duplicate graphs, representing equivalent functions, to be eliminated. Canonicity can be further exploited through *complement* and *swap* flags (the output and input complement flags of [6]), where a function and its complement, or a function and the function obtained by complementing its top input variable, are both represented in the same BDD. These improvements can each reduce the BDD size by at most a factor of 2, and can be combined to reduce the BDD size by at most a factor of 4.

The second factor leading to compact BDD representation is the elimination of the nodes that form certain patterns, producing “long edges” that skip over variables in the BDD. Bryant’s original BDD definition [7] eliminated *redundant* nodes, which occur when a function does not depend on

the variable specified by the node. Zero-suppressed binary decision diagrams (ZDDs) [8] instead eliminate nodes whose high edge is terminal zero, meaning that the given variable, if true, causes the function to evaluate to false. Recent work has focused on combining these forms, and extending the number of patterns that can be exploited and eliminated from the BDD representation. Two different proposed forms are *chain* based forms, including chained BDDs and chained ZDDs [9], and *edge* based forms including tagged BDDs [10], and RexBDDs [11]. Specifically, current state-of-the-art RexBDDs combine complement flags, swap flags, and nine reduction rules attributed to edges, to yield compact representations.

This paper proposes Unchained BDDs (UBDDs). Informally, UBDDs are based on replacing the chain nodes from chained BDDs with edge rules, and allowing both *and*-chains and *or*-chains. This allows UBDDs to exploit the same patterns as chained BDDs and RexBDDs, along with additional patterns that neither address. Thus, UBDDs can be seen as an extension and generalization of both chained BDDs and RexBDDs.

The remainder of the paper is organized as follows. Section II reviews some existing BDD variants. Section III formally introduces UBDDs, presents algorithms for their manipulation, and proves canonicity of UBDDs. Sections IV and V give theoretical and experimental comparisons between UBDDs and existing BDD variants, respectively. Finally, Section VI concludes the paper and gives some directions for future work.

II. BACKGROUND AND NOTATION

The following notations are used throughout the paper. The set of Booleans is denoted $\mathbb{B} = \{0, 1\}$. Operator $\oplus$ denotes exclusive-or, and the ternary operation $a ? b : c$ denotes “if a then b otherwise c ”. Given a function $f : \mathbb{B}^n \rightarrow \mathbb{B}$, we often write simply f instead of $f(x_n, \dots, x_1)$. We use operators on functions to denote element-wise operations; for example, $h = f \oplus g$ indicates that $h(x_n, \dots, x_1) = f(x_n, \dots, x_1) \oplus g(x_n, \dots, x_1)$. It is well-known [7] that BDDs and other variants can be extremely sensitive to the choice of variable ordering. For the entire paper, we consider only *ordered* variants of BDDs, and we assume that the variables assume a fixed order $x_n, \dots, x_1$ with x_n as the top-most variable.

A. Edge reduction rules

We recall the definition of (unreduced) reduction-on-edge complement-and-swap BDDs (RexBDDs) [11].

Definition 1: An L -level RexBDD is a directed acyclic graph (DAG) where:

- Each edge is a tuple $\langle \rho, c, s, q \rangle$, where $\rho \in \{X, EL_0, EL_1, EH_0, EH_1, AL_0, AL_1, AH_0, AH_1\}$ is a *reduction rule*, $c \in \mathbb{B}$ is a *complement flag*, $s \in \mathbb{B}$ is a *swap flag*, and q is the target node. If $e = \langle \rho, c, s, q \rangle$, we write $e.r = \rho$, $e.c = c$, $e.s = s$, and $e.d = q$.
- The terminal nodes are 0 and 1 , with $0.lvl = 1.lvl = 0$.
- Each nonterminal node p belongs to a level $p.lvl \in \{1, \dots, L\}$, and has children edges $p[0]$ (drawn as a dashed line) and $p[1]$ (drawn as a solid line). The ordering property requires $p.lvl > p[0].d.lvl$ and $p.lvl > p[1].d.lvl$.
- The function of n variables encoded by $e = \langle \rho, c, s, q \rangle$, with $k = q.lvl$ and $n \geq k$, is defined as

$$f_e^n \triangleq \begin{cases} c \oplus q & \text{if } n = k = 0 \\ c \oplus f_{q[s \oplus x_n]}^{n-1} & \text{if } n = k > 0 \\ f_e^k & \text{if } n > k \wedge \rho = X \\ (x_{k+1} \wedge \dots \wedge x_n) ? f_e^k : t & \text{if } n > k \wedge \rho = EL_t \\ (x_{k+1} \vee \dots \vee x_n) ? f_e^k : t & \text{if } n > k \wedge \rho = AL_t \\ (x_{k+1} \vee \dots \vee x_n) ? t : f_e^k & \text{if } n > k \wedge \rho = EH_t \\ (x_{k+1} \wedge \dots \wedge x_n) ? t : f_e^k & \text{if } n > k \wedge \rho = AH_t \end{cases}$$

Note that a nonterminal node at level k corresponds to a decision based on function variable x_k .

□

Given a nonterminal RexBDD node p , its complement, denoted $\bar{p}$, and its swap, denoted $\check{p}$, are defined as follows.

Definition 2: The complement of a RexBDD edge rule ρ , denoted $\bar{\rho}$, is defined as $\bar{X} \triangleq X$, $\bar{EL}_t \triangleq EL_{\bar{t}}$, $\bar{EH}_t \triangleq EH_{\bar{t}}$, $\bar{AL}_t \triangleq AL_{\bar{t}}$, $\bar{AH}_t \triangleq AH_{\bar{t}}$, where $t \in \mathbb{B}$. The complement of a RexBDD edge e , denoted $\bar{e}$, is defined as $\langle \bar{\rho}, \bar{c}, \bar{s}, \bar{p} \rangle \triangleq \langle \bar{\rho}, \bar{c}, \bar{s}, p \rangle$. The complement of a RexBDD nonterminal node p has $\bar{p}.lvl = p.lvl$, $\bar{p}[0] = p[0]$, and $\bar{p}[1] = p[1]$. The swap of a RexBDD nonterminal node p has $\check{p}.lvl = p.lvl$, $\check{p}[0] = p[0]$, and $\check{p}[1] = p[1]$.

To obtain a canonical form, RexBDDs must be *reduced*, using a carefully constructed set of restrictions to eliminate various patterns in the graph. Summarizing from [11], a RexBDD is reduced if it has:

- 1) No duplicate nodes. Nonterminal nodes p and q are *duplicates* if $p.lvl = q.lvl$, $p[0] = q[0]$, and $p[1] = q[1]$.
- 2) No *pattern* nodes, i.e., no nonterminal node p where p could be replaced by a long edge to one of p 's children. There are *thirteen* distinct types of pattern nodes that must be considered and eliminated. First, there are the nine patterns X , EL_0 , EL_1 , EH_0 , EH_1 , AL_0 , AL_1 , AH_0 , and AH_1 , corresponding to each edge rule. Then there are four special patterns, applicable only for edges pointing to 0 or 1 : C for constant functions $f = c$ where $c \in \mathbb{B}$, B for functions of the bottom variable $f = c \oplus x_1$, D for an “and chain” function of the form $f = c \oplus (x_k \wedge$

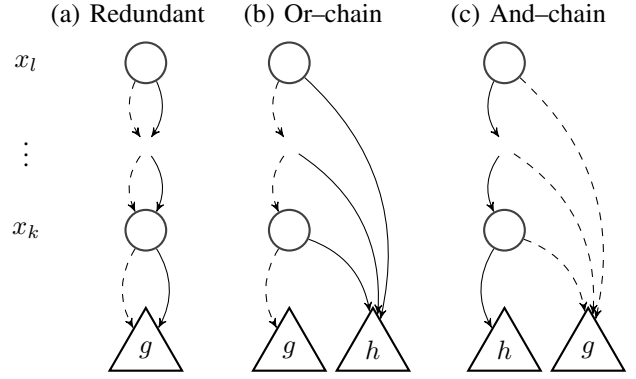


Fig. 1. Three sufficient patterns, where the function f encoded is (a) $f = g$; (b) $f = (x_k \vee \dots \vee x_l) ? h : g$; and (c) $f = (x_k \wedge \dots \wedge x_l) ? h : g$.

$\dots \wedge x_1)$, and R for an “or chain” function of the form $f = c \oplus (x_k \vee \dots \vee x_1)$.

The number and complexity of the rules are necessary to resolve overlaps between edge rules. For example, one can see from Def. 1 that edges $\langle EL_0, 0, 0, 1 \rangle$ and $\langle AH_1, 0, 0, 0 \rangle$ encode the same function; these are captured by the D pattern. We omit a formal discussion of the pattern definitions in the interest of space, and instead refer the interested reader to [11] for details.

- 3) No terminal 1 (use instead a complemented edge to 0).
- 4) No nonterminal node p with $p[0].c = 1$ (store $\bar{p}$ instead).
- 5) No “useless” swap flags: the swap flag cannot be set on any edge to 0 , or to a nonterminal node p when $\check{p} = p$, $\check{p} = \bar{p}$, or $\check{p}$ is a pattern node.
- 6) No nonterminal node p with $p[0].d > p[1].d$; or $p[0].d = p[1].d$, $p[0].s = 1$, and $p[1].s = 0$; or $p[0].d = p[1].d$, $p[0].s = p[1].s$, and $p[0].r > p[1].c \oplus p[1].r$, using some ordering over nodes and over reduction rules. However, the above restriction is dropped if $\check{p}$ is a pattern node.

Restriction 1 is common for all BDD variants considered in this paper, and originated with Bryant’s BDDs [7]. Restriction 2 is the generalization of Bryant’s *redundant* nodes. Restrictions 3 and 4 are common for BDDs with (output) complement flags [6]. Restriction 5 is required when combining complement and swap flags [12], and swap flags with edge rules. Finally, restriction 6 (along with 4) determines which one of p , $\check{p}$, $\bar{p}$, and $\check{\bar{p}}$ is kept in the RexBDD.

Note that each RexBDD reduction rule corresponds to one of the three patterns shown in Fig. 1. Specifically, rule X on an edge to node g corresponds to the redundant pattern in Fig. 1(a). Rules EH_0 and EH_1 on edges to node g correspond to the or-chain pattern in Fig. 1(b), where node h is terminal node zero or one, respectively. Rules AL_0 and AL_1 on edges to node h correspond to the or-chain pattern where node g is terminal node zero or one, respectively. Similarly, rules EL_0 , EL_1 , AH_0 , and AH_1 correspond to the and-chain pattern in Fig. 1(c), where exactly one of g or h is terminal node zero or one. RexBDDs are not able to exploit a more general or-chain or and-chain where both g and h are nonterminal nodes.

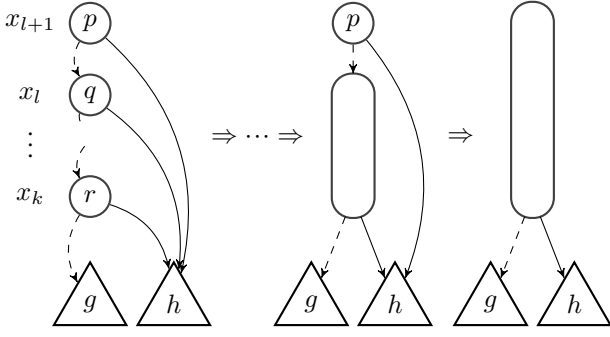


Fig. 2. Reducing an or-chain into a chain node in a chained BDD.

RexBDDs are an extension of CESRBDDs, which in turn extend ESRBDDs. CESRBDDs [13] can be viewed as the subset of RexBDDs containing only complement flags and edge rules $\{X, EL_0, EL_1, EH_0, EH_1\}$. ESRBDDs [14] can be viewed as the subset of RexBDDs without both complement and swap flags, and having only edge rules $\{X, EL_0, EH_0\}$. Bryant's original BDDs [7] can be viewed as the subset of RexBDDs having only edge rule X , without both complement and swap flags.

B. Chain nodes

An alternative to labeling edges with reduction rules is to instead utilize *chain nodes* [9], which utilize one reduction rule on all long edges, and instead capture one of the patterns in Fig. 1 within a chain node that spans multiple levels. Chained BDDs have long edges using rule X , and chain nodes represent the or-chain pattern in Fig. 1(b). Chained ZDDs instead have long edges using rule EH_0 (as done in zero-suppressed BDDs), and chain nodes represent the redundant pattern in Fig. 1(a). For the rest of the paper, we consider only chained BDDs.

Definition 3: An L -level chained BDD is a DAG where:

- For consistency with the other variants, each edge is a tuple $e = \langle q \rangle$ where $q = e.d$ is the target node.
- Every node p has a top level, $p.top$, and a bottom level, $p.bot$, satisfying $L \geq p.top \geq p.bot \geq 0$. If $p.top > p.bot$, then node p is drawn as an elongated oval.
- The terminal nodes are 0 and 1 , with $0.top = 0.bot = 1.top = 1.bot = 0$.
- Each nonterminal node p has $p.bot \geq 1$ and has children edges $p[0]$ and $p[1]$, drawn as dashed and solid lines, respectively. The ordering property requires $p.bot > p[0].d.top$ and $p.bot > p[1].d.top$.
- The function of n variables encoded by edge $e = \langle q \rangle$, with $n \geq q.bot$, is defined as $f_e^n \triangleq$

$$\begin{cases} q & \text{if } n = q.top = q.bot = 0 \\ f_e^{n-1} & \text{if } n > q.top \\ x_n ? f_{q[1]}^{n-1} : f_e^{n-1} & \text{if } q.top \geq n > q.bot > 0 \\ f_{q[x_n]}^{n-1} & \text{if } q.top \geq n = q.bot > 0 \end{cases} \quad \square$$

The reduction algorithm for chained BDDs [9] is based on transforming a reduced BDD as illustrated in Fig. 2. An ordinary nonterminal node p matches the or-chain pattern if

$p[0].top$ equals $p.bot - 1$, and $p[1]$ equals $p[0][1]$. In that case, node p can be replaced by a chain node. Additionally, duplicate nodes must be eliminated. Note that for chained BDDs, nodes p and q are duplicates if $p.top = q.top$, $p.bot = q.bot$, $p[0] = q[0]$, and $p[1] = q[1]$. Thus, the chained BDD in Fig. 2 requires different chain nodes for p , q , and r .

When the chain reduction is restricted to only zero chains (as done in ZDDs), where function $h = 0$ in Fig. 2, then chained BDDs are subsumed by ESRBDDs, as these also exploit the zero chain pattern. If instead the chain reduction is applied for arbitrary functions g and h , then ESRBDDs and chained BDDs are incomparable: chained BDDs can exploit generic or-chains, while ESRBDDs cannot; and ESRBDDs can exploit some and-chains, while chained BDDs cannot. Thus, there are functions where ESRBDDs are about a factor of L worse than chained BDDs, and vice-versa. Finally, as suggested in [9], complement flags could be integrated into chained BDDs. While swap flags are not mentioned, we suspect that those too could be integrated into chained BDDs.

III. UNCHAINED BDDs

We are now ready to introduce *Unchained BDDs* (UBDDs). The primary goal is to create a variant of BDD that can exploit the general form of all three patterns shown in Fig. 1.

A. Unreduced UBDDs

Definition 4: An L -level Unchained BDD is a DAG where

- Each edge is a tuple $\langle \rho, c, s, q \rangle$, where $\rho \in \{N, X, And, Or\}$ is a reduction rule, $c \in \mathbb{B}$ is a complement flag, $s \in \mathbb{B}$ is a swap flag, and q is the target node. If $e = \langle \rho, c, s, q \rangle$, then we write $e.r = \rho$, $e.c = c$, $e.s = s$, and $e.d = q$.
- Terminal nodes are 0 and 1 , with $0.lvl = 1.lvl = 0$.
- Each nonterminal node p belongs to a level $p.lvl \in \{1, \dots, L\}$ and has children edges $p[0]$ and $p[1]$, drawn as dashed and solid lines, respectively. The ordering property requires $p.lvl > p[0].d.lvl$ and $p.lvl > p[1].d.lvl$.
- An edge is said to be *short* if it is a root edge, i.e., an edge originating from the level L , to a node at level L , or if it originates from a node at level $n + 1$ and has a target node at level n . Edge e has rule $e.r = N$ if and only if e is short.
- Edges to a terminal node must use reduction rule N or X , and cannot have the swap flag set.
- The function of n variables encoded by $e = \langle \rho, c, s, q \rangle$, with $n \geq q.lvl = k$, is defined as $f_e^n \triangleq$

$$\begin{cases} c \oplus q & \text{if } n \geq k = 0 \\ c \oplus f_{q[s \oplus x_n]}^{n-1} & \text{if } n = k > 0 \\ f_e^{n-1} & \text{if } n > k \wedge \rho = X \\ x_n ? f_e^{n-1} : c \oplus f_{q[s]}^{n-1} & \text{if } n > k \wedge \rho = And \\ x_n ? c \oplus f_{q[\bar{s}]}^{n-1} : f_e^{n-1} & \text{if } n > k \wedge \rho = Or \end{cases} \quad \square$$

Conceptually, UBDDs are chained BDDs with support for both and-chains and or-chains. Instead of using chain nodes, though, UBDDs utilize edge rules. This has several benefits. First, chain nodes that are identical except for their top levels are represented by the same node. Second, and-chain nodes,

or-chain nodes, and ordinary nodes with the same children are represented by the same node. Finally, a chain node requires an additional integer to store the top level, while edge rules instead require only a few additional bits per edge. Additionally, we note that our or-chains are more general than the ones shown in Fig. 1(b) and utilized by chained BDDs: the edges into function h may have any edge rule (not just X) and any flag settings, as long as they are the same edge.

Complement and swap flags serve their usual purposes: toggling the complement flag inverts the output of the encoded function, and setting the swap flag exchanges the children of the target (nonterminal) node. In the case of N or X edges, this inverts the variable of the target node. In the case of And and Or edges, this exchanges the g and h subtrees at the bottom of the and- or or-chain shown in Fig. 1. We formalize these properties with the following definitions and theorems.

Definition 5: Given UBDD edge $e = \langle \rho, c, s, q \rangle$ and Boolean $b \in \mathbb{B}$, $b \oplus e \triangleq \langle \rho, b \oplus c, s, q \rangle$. The complement of UBDD edge e is $\bar{e} \triangleq 1 \oplus e$, and the complement of UBDD nonterminal node p , denoted $\bar{p}$, has $\bar{p}.lvl = p.lvl$, $\bar{p}[0] = 1 \oplus (p[0])$, and $\bar{p}[1] = 1 \oplus (p[1])$. $\square$

Theorem 1: For any UBDD edge $e = \langle \rho, c, s, q \rangle$, $b \in \mathbb{B}$, and integer $n \geq q.lvl$, $f_{b \oplus e}^n = b \oplus f_e^n$.

Proof of Theorem 1: We use induction on n and enumerate the cases of definition 4 (where $k = q.lvl$):

- $n \geq k = 0$: $f_{b \oplus e}^n = b \oplus c \oplus q = b \oplus f_e^n$
- $n = k > 0$: $f_{b \oplus e}^n = b \oplus c \oplus f_{q[s \oplus x_n]}^{n-1} = b \oplus f_e^n$
- $n > k \wedge \rho = X$: $f_{b \oplus e}^n = f_{b \oplus e}^{n-1} = b \oplus f_e^{n-1} = b \oplus f_e^n$
- $n > k \wedge \rho = \text{And}$:

$$\begin{aligned} f_{b \oplus e}^n &= x_n ? f_{b \oplus e}^{n-1} : b \oplus c \oplus f_{q[s]}^{n-1} \\ &= x_n ? b \oplus f_e^{n-1} : b \oplus c \oplus f_{q[s]}^{n-1} = b \oplus f_e^n \end{aligned}$$

- $n > k \wedge \rho = \text{Or}$: similar to the previous case $\square$

Definition 6: The *swap* of nonterminal UBDD node p , denoted $\check{p}$, has $\check{p}.lvl = p.lvl$ and $\check{p}[b] = p[\bar{b}]$ for $b \in \mathbb{B}$. $\square$

Theorem 2: For any edge $\langle \rho, c, s, q \rangle$ where q is a nonterminal node, $f_{\langle \rho, c, \bar{s}, q \rangle}^n = f_{\langle \rho, c, s, \check{q} \rangle}^n$.

Proof of Theorem 2: The proof is similar to the proof of Theorem 1, and is omitted for space. $\square$

Note that the node swap definitions are the same for RexBDDs and UBDDs (cf. Defs. 2 and 6), while the edge complement definitions are different (cf. Defs. 2 and 5): for a RexBDD edge, $\bar{e}$ requires toggling both the edge rule and the complement flag of e , while for a UBDD edge, $\bar{e}$ toggles only the complement flag of e .

B. Reduction rules and canonicity

Additional rules are necessary to *reduce* the size of a UBDD representation of a given function, and to ensure a canonical form. Informally, the reduction rules serve to eliminate any duplication in the graph, eliminate any patterns that can instead be represented by long edges, and ensure that, for a nonterminal node p , at most one of p , $\bar{p}$, $\check{p}$, and $\check{\bar{p}}$ is stored in the graph, similar to the reduction rules for RexBDDs and

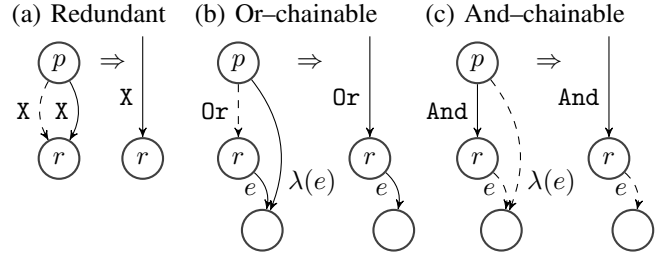


Fig. 3. Eliminating redundant, or-chainable, and and-chainable nodes.

other BDD variants. Before presenting the UBDD reduction rules, we first formally define some terms.

Definition 7: The *lengthening* of edge e , defined as

$$\lambda(e) \triangleq \langle e.r = N ? X : e.r, e.c, e.s, e.d \rangle$$

extends e by changing rule N to rule X. $\square$

Definition 8: Within a UBDD, nonterminal nodes p and q are *duplicates* if $p.lvl = q.lvl$, $p[0] = q[0]$, and $p[1] = q[1]$. $\square$

Definition 9: Within a UBDD, nonterminal node p is

- *redundant* if $p[0] = p[1]$ and $p[0].r \in \{N, X\}$;
- *and-chainable* if $p[1].r \in \{N, \text{And}\}$, $p[1].d$ is nonterminal, and $p[0] = \lambda(p[1].c \oplus p[1].d[p[1].s])$;
- *or-chainable* if $p[0].r \in \{N, \text{Or}\}$, $p[0].d$ is nonterminal, and $p[1] = \lambda(p[0].c \oplus p[0].d[1 \oplus p[0].s])$. $\square$

Fig. 3 illustrates examples of each of the pattern nodes of Def. 9, and how they may be replaced by long edges. Note that Fig. 3 illustrates the easy case when no complement or swap flags are set; the somewhat more complex Def. 9 correctly handles those cases as well.

In addition to eliminating pattern nodes, we must ensure that a node and its swap are not both part of the BDD. This is done by defining an ordering $\succ$ over edges that ignores complement flags. Any total ordering over edges with complement flags fixed at 0 will work; for example, one can first define a total ordering on nodes (based on storage addresses), and in case of a tie, compare the swap flags, and in case of a tie, use a total ordering on the reduction rules.

Definition 10: Nonterminal node p is *normalized* if $p[0].c = 0$, and $p[0] \preceq p[1]$ or $\check{p}$ is and-chainable or or-chainable. $\square$

Note that only one of p , $\bar{p}$, $\check{p}$, and $\check{\bar{p}}$ is normalized.

Definition 11: A UBDD is *reduced* if the following hold.

- 1) There is no terminal node 1.
- 2) For any edge $\langle \rho, c, s, q \rangle$, if $\check{q} = q$, $\check{q} = \bar{q}$, $\check{q}$ is and-chainable, or $\check{q}$ is or-chainable, then the swap flag s must be 0.
- 3) There are no duplicate, redundant, and-chainable, or or-chainable nodes.
- 4) All nonterminal nodes are normalized. $\square$

Theorem 3: Given a reduced UBDD, integer $n \geq 0$, and two edges $e = \langle \rho, c, s, q \rangle$ and $e' = \langle \rho', c', s', q' \rangle$ with $q.lvl \leq n$ and $q'.lvl \leq n$, if $f_e^n = f_{e'}^n$, then $c = c'$, $s = s'$, $q = q'$. Additionally, if $q.lvl < n$, then $\rho = \rho'$.

Proof of Theorem 3 (sketch): We use proof by induction on n . In the base case $n = 0$, it follows that $q = q' = 0$, and

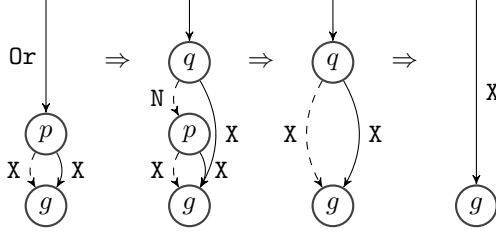


Fig. 4. Applying REDUCEEDGE with $\rho = \text{Or}$ on redundant node p .

therefore $s = s' = 0$. From Def. 4 we have $c \oplus q = c' \oplus q'$ which implies $c = c'$.

For the inductive step, assume $q'.lvl \leq q.lvl$, and first consider the case $n = q.lvl$. We first establish $n = q'.lvl$ by contradiction: a long edge to q' with edge rule X, And, or Or will imply that q is redundant, and-chainable, or or-chainable (or, that $s = 1$ and $\tilde{q}$ is and-chainable or or-chainable). Then, from Def. 4 and the inductive hypothesis, we obtain $c \oplus q[s \oplus x_n] = c' \oplus q'[s' \oplus x_n]$ for $x_n = 0$ and $x_n = 1$. We can then show that $c = c'$ and $s = s'$ by contradiction, utilizing the properties of normalization. It then follows that $q[0] = q'[0]$ and $q[1] = q'[1]$, implying $q = q'$ since duplicates are not allowed.

For the case $n > q.lvl$, we enumerate the possibilities for edge rule ρ , first establishing that $\rho = \rho'$ by contradiction (all cases for $\rho \neq \rho'$ imply that some child node is not below its parent, violating the ordering property). Then from Def. 4 we have $f_e^{n-1} = f_{e'}^{n-1}$, and the inductive hypothesis gives us $q = q'$, $c = c'$, $s = s'$. $\square$

C. Reduction operations on UBDDs

Construction of UBDDs follows the same basic idea as that of other variants of BDDs, namely, UBDDs tend to be constructed in a bottom-up fashion, often using recursion. Temporary nodes, each encoding a desired function, are built and then added to the UBDD in such a way as to ensure the overall UBDD remains reduced. This involves detecting and eliminating pattern nodes, as shown in Fig. 3. We provide a more general procedure, for temporary *edge* reduction, detailed in Alg. 1. Procedure REDUCEEDGE takes an edge rule ρ to a temporary node p whose children edges are already reduced, and returns an equivalent edge e such that $f_e^n = f_{\langle \rho, 0, 0, p \rangle}^n$ and the resulting edge satisfies the reduction requirements in Def. 11. In addition to eliminating the patterns of Fig. 3, procedure REDUCEEDGE must handle an edge rule ρ to an eliminated pattern. If the resulting edge after eliminating the pattern is $\langle \rho', c', s', p' \rangle$, then whenever $\rho \neq \rho'$ and $\rho' \neq \text{X}$, an extra node must be created at the level above node p ; this is handled by procedure MAKEABOVE. Otherwise, we say ρ is *compatible* with p , and no extra node is required.

For a redundant node p , all incoming edge rules are compatible. Fig. 4 illustrates an example with $\rho = \text{Or}$; the case $\rho = \text{And}$ is similar. In the figure, node q at the bottom of the or-chain is expanded. After eliminating node p , node q becomes redundant. This process can be repeated all the way

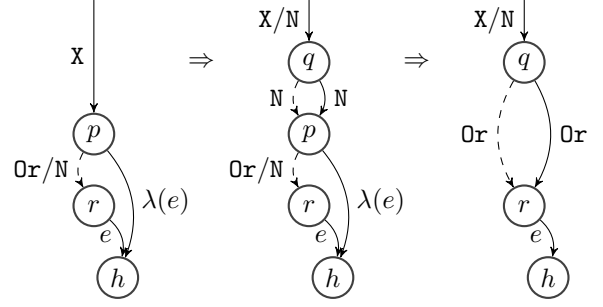


Fig. 5. Applying REDUCEEDGE with $\rho = \text{X}$ on or-chainable node p .

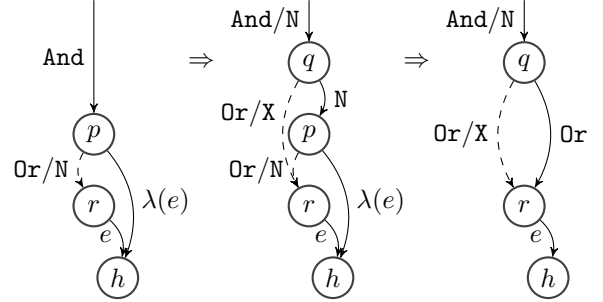


Fig. 6. Applying REDUCEEDGE with $\rho = \text{And}$ on or-chainable node p .

up the incoming Or edge, ultimately producing a long X edge. This case is handled in lines 3–4 of Alg. 1. Note that node p is eliminated, and no new node is added to the UBDD.

The case of eliminated or-chainable nodes is handled in lines 5–9. Incoming edge rules N and Or are compatible, and cause node p to be eliminated. The case of incoming edge rule X is illustrated in Fig. 5, and shows that new node q , expanded from the bottom of the X edge rule, has two outgoing Or edges corresponding to eliminated node p . The case of incoming edge rule And is illustrated in Fig. 6, and shows that new node q , expanded as an and-chain directly above node p , has a high Or edge corresponding to eliminated node p , and a low Or or X edge due to the and-chain.

The case of eliminated and-chainable nodes is handled in lines 10–14, and is similar to the or-chainable case.

The remaining lines handle the case when p is not a pattern node. Lines 15–18 determine if node p or node $\tilde{p}$ belong in the UBDD, as part of the normalization process. Then, line 19 ensures that the low edge of the stored node is not complemented. Line 20 checks to see if a duplicate of node p already exists in the UBDD. Procedure INSERTUNIQUE, not given, checks the unique table (typically, a hash table) for a duplicate of node p . If found, the existing node is returned and node p is discarded. Otherwise, a permanent copy of node p is added to the unique table and returned.

D. Conjunction operations on UBDDs

We now present algorithms to construct the conjunction of two functions. Specifically, given edges e and e' in a reduced UBDD, we would like to construct edge a such that $f_a^n =$

Algorithm 1 Returns reduced e such that $f_e^n = f_{\langle \rho, 0, 0, p \rangle}^n$

```

1: procedure REDUCEEDGE(level  $n$ , rule  $\rho$ , node  $p$ )
2:   if  $p.lvl = 0$  then return  $\langle n > 0 ? X : N, p = 1, 0, 0 \rangle$ 
3:   if REDUNDANT( $p$ ) then
4:     return  $\langle X, p[0].c, p[0].s, p[0].d \rangle$ 
5:   else if ORCHAIN( $p$ ) then
6:      $e \leftarrow p[0]$ ;  $e.r \leftarrow 0r$ 
7:     if  $\rho \in \{N, 0r\}$  then return  $e$ 
8:     if  $\rho = X$  then  $p \leftarrow \text{MAKEABOVE}(p, e, e)$ 
9:     else  $p \leftarrow \text{MAKEABOVE}(p, p[0], e)$ 
10:  else if ANDCHAIN( $p$ ) then
11:     $e \leftarrow p[1]$ ;  $e.r \leftarrow \text{And}$ 
12:    if  $\rho \in \{N, \text{And}\}$  then return  $e$ 
13:    if  $\rho = X$  then  $p \leftarrow \text{MAKEABOVE}(p, e, e)$ 
14:    else  $p \leftarrow \text{MAKEABOVE}(p, e, p[1])$ 
15:  if  $p[0] \preceq p[1] \vee \text{ORCHAIN}(\check{p}) \vee \text{ANDCHAIN}(\check{p})$  then
16:     $s \leftarrow 0$ 
17:  else
18:     $s \leftarrow 1$ ;  $p \leftarrow \check{p}$ 
19:   $c \leftarrow p[0].c$ ;  $p[0] \leftarrow c \oplus p[0]$ ;  $p[1] \leftarrow c \oplus p[1]$ ;
20:   $p \leftarrow \text{INSERTUNIQUE}(\check{p})$ 
21:  return  $\langle n > p.lvl ? \rho : N, c, s, p \rangle$ 
22: procedure MAKEABOVE(node  $p$ , edge  $e0$ , edge  $e1$ )
23:   $q \leftarrow$  new node at level  $p.lvl + 1$ 
24:   $q[0] \leftarrow \lambda(e0)$ ;  $q[1] \leftarrow \lambda(e1)$ 
25:  return  $q$ 

```

Algorithm 2 Returns edge encoding $f_{\langle \rho, c, s, q \rangle}^n|_{x_n=i}$

```

1: procedure COFACT(level  $n$ , edge  $\langle \rho, c, s, q \rangle$ , bool  $i$ )
2:   if  $n = q.lvl$  then return  $c \oplus q[s \oplus i]$ 
3:   if  $\rho = \text{And} \wedge i = 0$  then return  $c \oplus q[s]$ 
4:   if  $\rho = 0r \wedge i = 1$  then return  $c \oplus q[\bar{s}]$ 
5:   return  $\langle \rho, c, s, q \rangle$ 

```

$f_e^n \wedge f_{e'}^n$. The algorithms to construct the disjunction of two functions are similar; alternatively, one could use complementation (which requires only to toggle the complement flags) and conjunction instead: $f_e^n \vee f_{e'}^n = 1 \oplus ((1 \oplus f_e^n) \wedge (1 \oplus f_{e'}^n))$. Our algorithms are similar to those for other variants of BDDs, in that they traverse the operands recursively by levels, and the UBDD is kept reduced during construction using procedure REDUCEEDGE discussed earlier.

We first present a straightforward algorithm that proceeds recursively by levels, based on the cofactor decomposition $f_a^n|_{x_n=i} = f_e^n|_{x_n=i} \wedge f_{e'}^n|_{x_n=i}$. We utilize a helper procedure, COFACT, shown in Alg. 2 to determine cofactor $x_n = i$ for a function encoded by a given edge. The algorithm follows trivially from Def. 4 by substituting $x_n = i$. Procedure LEVELAPPLYAND, shown in Alg. 3, recursively constructs $f_e^n \wedge f_{e'}^n$, where n is decreased by exactly one with each recursive call.

Lines 3–7 handle terminal cases for the recursion. Line 3

Algorithm 3 Determine a such that $f_a^n = f_e^n \wedge f_{e'}^n$

```

1: procedure LEVELAPPLYAND(level  $n$ , edge  $e$ , edge  $e'$ )
2:    $z \leftarrow \langle n > 0 ? X : N, 0, 0, 0 \rangle$ 
3:   if  $\bar{e} = e' \vee e = z \vee e' = z$  then return  $z$ 
4:   if  $e = e' \vee e' = \bar{z}$  then return  $e$ 
5:   if  $e = \bar{z}$  then return  $e'$ 
6:   if  $e \prec e'$  then SWAP( $e, e'$ )
7:   if  $((\wedge, n, e, e'), a) \in \text{cache}$  then return  $a$ 
8:    $p \leftarrow$  new node at level  $n$ 
9:   for  $i \in \mathbb{B}$  do
10:     $e_i \leftarrow \text{COFACT}(n, e, i)$ ;  $e'_i \leftarrow \text{COFACT}(n, e', i)$ 
11:     $p[i] \leftarrow \text{LEVELAPPLYAND}(n-1, e_i, e'_i)$ 
12:    $a \leftarrow \text{REDUCEEDGE}(n, N, p)$ 
13:   add  $((\wedge, n, e, e'), a)$  to cache
14:   return  $a$ 

```

handles trivial cases where the result is the constant zero function: $f_e^n \wedge f_{e'}^n$, $f_e^n \wedge 0$, and $0 \wedge f_{e'}^n$. Note that checking if two functions are the complement of each other is equivalent to checking if their edges are complements of each other. Line 4 handles $f_e^n \wedge f_{e'}^n$, and $f_e^n \wedge 1$ (the constant one function), both of which produce f_e^n . Line 5 handles $1 \wedge f_{e'}^n$. Line 6 reorders edges to exploit commutativity: $f_{e'}^n \wedge f_e^n = f_e^n \wedge f_{e'}^n$. Line 7 checks to see if the result $f_e^n \wedge f_{e'}^n$ has already been constructed, which is critical to reduce unnecessary duplication in computation. Lines 8–13 perform actual computation, with line 13 handling memoization.

While procedure LEVELAPPLYAND is correct, it is potentially inefficient, as it does not take advantage of any long edges. If both operands have long X edges, the resulting edge will be to a node whose level is the top-most level of its operands, just like for BDD operations [7]. Certain other combinations of long edges are also “skippable”, which can be seen by examining the patterns shown in Fig. 1. Combining a redundant chain with an or-chain or and-chain produces another or-chain or and-chain, respectively, but note that if the redundant chain is not longer than the or-chain or and-chain, then the resulting chain will be one level shorter than the redundant chain. This is because or-chains and and-chains are represented by long edges to the node at the bottom of the chain, while redundant chains are represented by long edges to the node *below* the chain. Combining two or-chains will produce another or-chain, but only if at least one of the or-chains has rule X on its long edges to subtree h in Fig. 1(b). More precisely, at least one of $e.d[1 \oplus e.s].r$ and $e'.d[1 \oplus e'.s].r$ must be X. Similarly, combining two and-chains will produce another and-chain, if at least one of $e.d[e.s].r$ and $e'.d[e'.s].r$ is X. Table I summarizes these rules; any case not listed should instead proceed one level at a time. Procedure APPLYAND, detailed in Alg. 4, is a modified version of procedure LEVELAPPLYAND that takes the rules of Table I into account (lines 8–20) to skip levels.

TABLE I
SKIPPING LEVELS IN APPLYAND ON INPUTS e, e' AND RESULT a .

Inputs		Output		
$e.r$	$e'.r$	$a.r$	$a.d.lvl$	when
X	X	X	$\max(e.d.lvl, e'.d.lvl)$	always
X	And	And	$\max(1 + e.d.lvl, e'.d.lvl)$	always
X	Or	Or	$\max(1 + e.d.lvl, e'.d.lvl)$	always
And	X	And	$\max(e.d.lvl, 1 + e'.d.lvl)$	always
And	And	And	$\max(e.d.lvl, e'.d.lvl)$	child X
Or	X	Or	$\max(e.d.lvl, 1 + e'.d.lvl)$	always
Or	Or	Or	$\max(e.d.lvl, e'.d.lvl)$	child X

Algorithm 4 Level skipping version of Alg. 3

```

1: procedure APPLYAND(level  $n$ , edge  $e$ , edge  $e'$ )
2:    $z \leftarrow \langle n > 0 ? X : N, 0, 0, 0 \rangle$ 
3:   if  $e = \bar{e}' \vee e = z \vee e' = z$  then return  $z$ 
4:   if  $e = e' \vee e' = \bar{z}$  then return  $e$ 
5:   if  $e = \bar{z}$  then return  $e'$ 
6:   if  $e \prec e'$  then SWAP( $e, e'$ )
7:   if  $((\wedge, n, e, e'), a) \in \text{cache}$  then return  $a$ 
8:    $\rho \leftarrow N; k \leftarrow n$ 
9:   if  $e.r = X \wedge e'.r = X$  then
10:     $\rho \leftarrow X; k \leftarrow \max(e.d.lvl, e'.d.lvl)$ 
11:   if  $e.r = X \wedge e'.r \in \{\text{And}, \text{Or}\}$  then
12:     $\rho \leftarrow e'.r; k \leftarrow \max(1 + e.d.lvl, e'.d.lvl)$ 
13:   if  $e.r \in \{\text{And}, \text{Or}\} \wedge e'.r = X$  then
14:     $\rho \leftarrow e.r; k \leftarrow \max(e.d.lvl, 1 + e'.d.lvl)$ 
15:   if  $e.r = \text{And} \wedge e'.r = \text{And}$  then
16:     if  $e.d[e.s].r = X \vee e'.d[e'.s].r = X$  then
17:        $\rho \leftarrow \text{And}; k \leftarrow \max(e.d.lvl, e'.d.lvl)$ 
18:   if  $e.r = \text{Or} \wedge e'.r = \text{Or}$  then
19:     if  $e.d[1 \oplus e.s].r = X \vee e'.d[1 \oplus e'.s].r = X$  then
20:        $\rho \leftarrow \text{Or}; k \leftarrow \max(e.d.lvl, e'.d.lvl)$ 
21:    $p \leftarrow \text{new node at level } k$ 
22:   for  $i \in \mathbb{B}$  do
23:      $e_i \leftarrow \text{COFACT}(n, e, i); e'_i \leftarrow \text{COFACT}(n, e', i)$ 
24:      $p[i] \leftarrow \text{APPLYAND}(k - 1, e_i, e'_i)$ 
25:    $a \leftarrow \text{REDUCEEDGE}(n, \rho, p)$ 
26:   add  $((\wedge, n, e, e'), a)$  to cache
27:   return  $a$ 

```

IV. THEORETICAL COMPARISON

We now compare several existing BDD variants with UBDDs, considering worst-case increases in number of nodes when converting to and from UBDDs. The results for this section are summarized in Table II. Note that UBDDs are never more than a constant factor larger than the other variants, while the reverse is not true.

We first consider conversion to UBDDs. Starting from a BDD, we can first build a (possibly unreduced) UBDD by copying the BDD but setting short edges to N and long edges to X . Reducing the UBDD from the bottom up will eliminate any and-chainable or or-chainable node p . In the worst case, all such nodes have an incoming long edge, and one new node

TABLE II
WORST-CASE INCREASES WHEN CONVERTING TO AND FROM UBDDs.

Variant	Variant $\Rightarrow$ UBDD	UBDD $\Rightarrow$ Variant
BDD	$1 \times$	$2L/3 \times$
ZDD	$2 \times$	$2L/3 \times$
Chained BDD	$2 \times$	$2L/3 \times$
Tagged	$5 \times$	$2L/3 \times$
Rex	$3 \times$	$L/2 \times$

must be created at the level above p ; this is illustrated for one or-chainable node in Fig. 7(a). The resulting UBDD will have the same number of nodes as the original BDD, so the worst-case increase is a factor of $1 \times$.

Starting from a ZDD, we again copy its nodes into a UBDD, but any node p with at least one long edge pointing to it in the ZDD will require addition of one node as the bottom of the or-chain with high edge to terminal zero, as illustrated in Fig. 7(b). In the worst case, this must be done for every node, leading to a $2 \times$ increase.

Starting from a chained BDD, consider a collection of nodes that would be duplicates except their top levels are different (for example, nodes p, q , and r in Fig. 8). In the corresponding UBDD, we must create one node corresponding to the bottom level of the chain, and one node for each node in the collection whose top level is above its bottom level, with Or edges to the bottom node, as shown in Fig. 8. If there is a chained BDD node p with $p.\text{top} = p.\text{bot}$, then the m chained BDD nodes will require m UBDD nodes. Otherwise, the m chained BDD nodes will require $m+1$ UBDD nodes. In the worst case, every such collection of nodes in the chained BDD is of size $m = 1$ and contains only a node p with $p.\text{top} > p.\text{bot}$, and the UBDD will have $2 \times$ the number of chained BDD nodes. If instead every collection contains one node p with $p.\text{top} = p.\text{bot}$, and no other reductions occur (there are no and-chainable nodes, no merging of p and $\bar{p}$, etc.) then the UBDD will have $1 \times$ the number of chained BDD nodes. Of course, these UBDD nodes are smaller than chained BDD nodes, as chained BDD nodes require an extra integer to represent the node's top level.

In a tagged BDD [10], each edge is tagged with a level, where the BDD reduction (X rule) is assumed from the tag upwards, and the ZDD reduction (EH_0 rule) is assumed below the tag. If both rules are utilized (i.e., the tag is between the source and target levels), we must introduce two UBDD nodes as shown in Fig. 7(c). As each node has two outgoing edges, in the worst case we must add four additional UBDD nodes for each tagged BDD node, leading to a $5 \times$ increase in nodes. Again, note that the UBDD nodes are smaller than the tagged BDD nodes, as each *edge* in the tagged BDD must store an additional integer for the tag, meaning two additional integers are required for each tagged BDD node.

Starting from a RexBDD, all long edges except X to a node p require addition of a UBDD node. However, at most *two* nodes will be created, as shown in Fig. 7(d): p_0 , for edge rules EH_0 , EL_0 , AH_0 , and AL_0 , and p_1 , for edge rules EH_1 , EL_1 , AH_1 , and AL_1 , with different incoming edge rules and swap flags for each Rex rule. This gives a worst-case increase of $3 \times$. In the

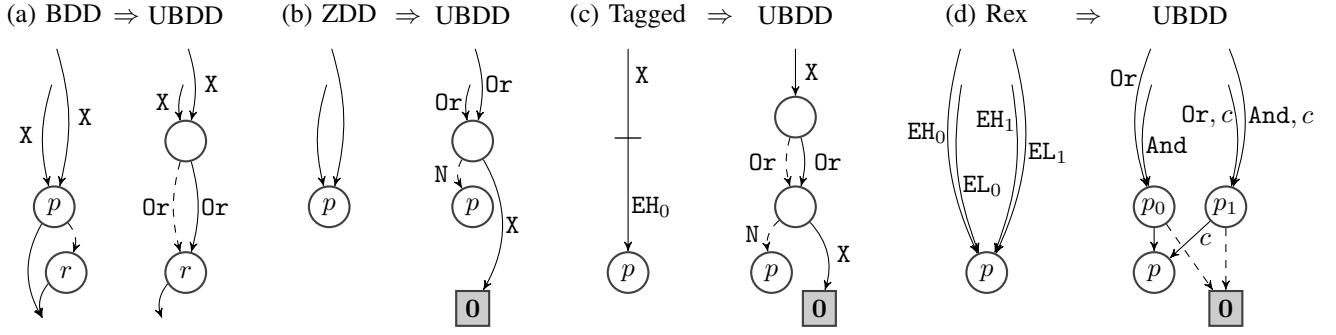


Fig. 7. Converting from other variants to UBDDs.

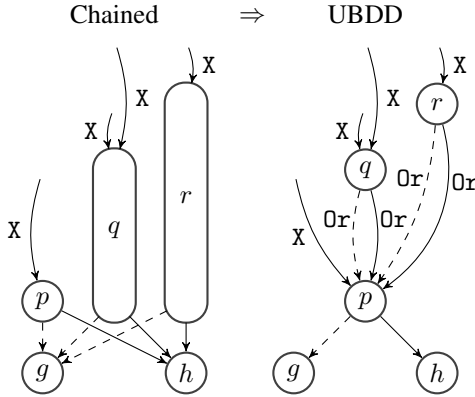


Fig. 8. Converting from chained BDDs to UBDDs.

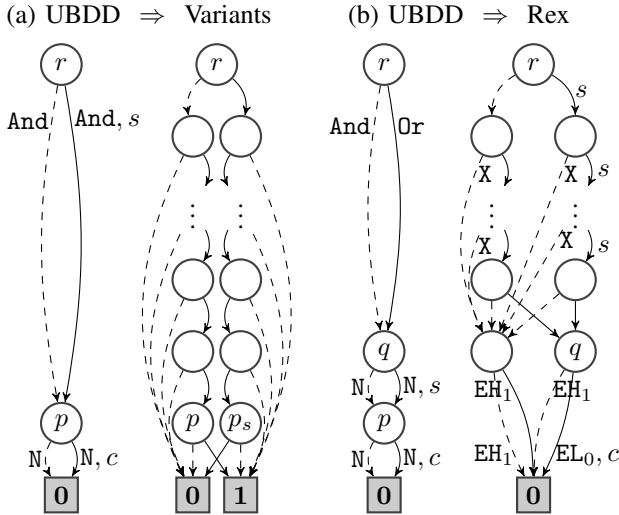


Fig. 9. Some worst-case conversions from UBDDs to other variants.

special case where each node p has no incoming edges with rule EH_1 , EL_1 , AH_1 , or AL_1 (corresponding to ESRBDDs), then the worst-case increase would be $2\times$.

Working in the other direction, a UBDD with long And or Or edges that are not supported in the target variant would require unfolding the entire and-chain or or-chain. Fig. 9(a) gives a worst-case example for all variants except RexBDDs: there is one node at the top level, with two long And edges to a

node at the bottom level, and the terminal node. This produces a BDD with $2L + 1$ nodes, an increase of about a factor of $2L/3$. The resulting chained BDD, tagged BDD, and ZDD would be identical, except the ZDD would eliminate node p_s . A similar example, shown in Fig. 9(b), can be used to produce a RexBDD that unrolls long edges, with a factor of increase of about $L/2$.

V. EXPERIMENTAL COMPARISON

For experimental comparison, we utilize our prototype implementation of UBDDs, built by modifying *BraveDD* [15], an open-source BDD library with support for BDDs, ZDDs, ESRBDDs, CESRBDDs, and RexBDDs. We utilized the provided implementation for RexBDDs, which should match the algorithms described in [11]. We could not easily add a native implementation of chained BDDs due to limitations of how *BraveDD* stores nodes internally (we could not store both a top and bottom level), so instead we used UBDDs restricted to reduction rules X and Or only, and with complement and swap flags both disabled, to roughly simulate chained BDDs. In the experimental results, we refer to these as *synthetic-chained* BDDs or *scBDDs*. From the earlier discussion in Sec. IV, the number of nodes in a scBDD is between $1\times$ and $2\times$ the nodes in a chained BDD, and timing results could be quite different, as we are not attempting to use chained BDD manipulation algorithms [9]. Nevertheless, we include scBDD results as a proxy for chained BDDs, to at least get a rough comparison.

For our experiments, we considered two categories of benchmark models: circuit synthesis and state space generation, discussed further below. We ran experiments for BDDs, ZDDs, CESRBDDs, RexBDDs, scBDDs, and UBDDs. Consistent with the findings in [11], we found that BDDs and ZDDs never outperform RexBDDs, and the results for CESRBDDs are similar to those of RexBDDs for circuits, and nearly identical to those of RexBDDs for state space generation. Thus, for brevity, we report only the results for scBDDs, RexBDDs, and UBDDs. Experiments were run on a high-performance cluster, with each benchmark limited to one thread and 128Gb of RAM.

For each benchmark, we record the *peak* number of nodes required for the computation, the *final* number of nodes required to represent the desired result, and the time required

TABLE III
EXPERIMENTAL RESULTS FOR CIRCUITS

Circuit	L	scBDD / UBDD			RexBDD / UBDD			UBDD		
		Peak	Final	Time	Peak	Final	Time	#Peak	#Final	Time (s)
too_large.blif	38	1.069×	1.169×	1.054×	0.886×	0.760×	0.990×	39606	4878	3.36
dalublif	75	1.572×	1.147×	1.275×	0.889×	0.904×	0.964×	11212	2323	1.25
C432.blif	36	1.470×	1.011×	1.182×	0.963×	0.975×	0.985×	15590	3959	0.82
i8.blif	133	1.043×	1.043×	1.110×	0.983×	0.981×	1.077×	13392	5385	3.47
C499.blif	41	1.935×	1.936×	1.237×	0.986×	0.992×	0.972×	73900	61932	9.40
C1355.blif	41	1.937×	1.936×	1.166×	0.942×	0.992×	0.958×	247244	61932	19.26
comp.blif	32	2.166×	1.898×	1.207×	1.050×	0.994×	0.978×	359343	241693	23.11
C5315.blif	178	1.453×	1.476×	1.361×	1.001×	1.006×	1.158×	112640	49277	12.64
rot.blif	135	1.078×	1.089×	1.072×	1.026×	1.025×	1.077×	165591	85693	28.06
frg1.blif	28	1.043×	1.032×	0.976×	1.036×	1.032×	0.990×	30106	9396	0.93
C880.blif	60	1.921×	1.046×	1.790×	1.033×	1.035×	0.977×	622727	452651	35.62
my_adder.blif	33	1.264×	1.324×	1.224×	0.956×	1.103×	0.960×	1063417	297099	63.74
pair.blif	173	2.062×	1.262×	1.870×	1.232×	1.161×	1.235×	61423	41542	19.20
C2670.blif	233	2.412×	2.219×	2.205×	1.198×	1.260×	1.049×	19627784	14878134	1323.67
Score	N/A	1.583	1.390	1.323	1.039	1.042	1.141	1.030	1.032	1.018

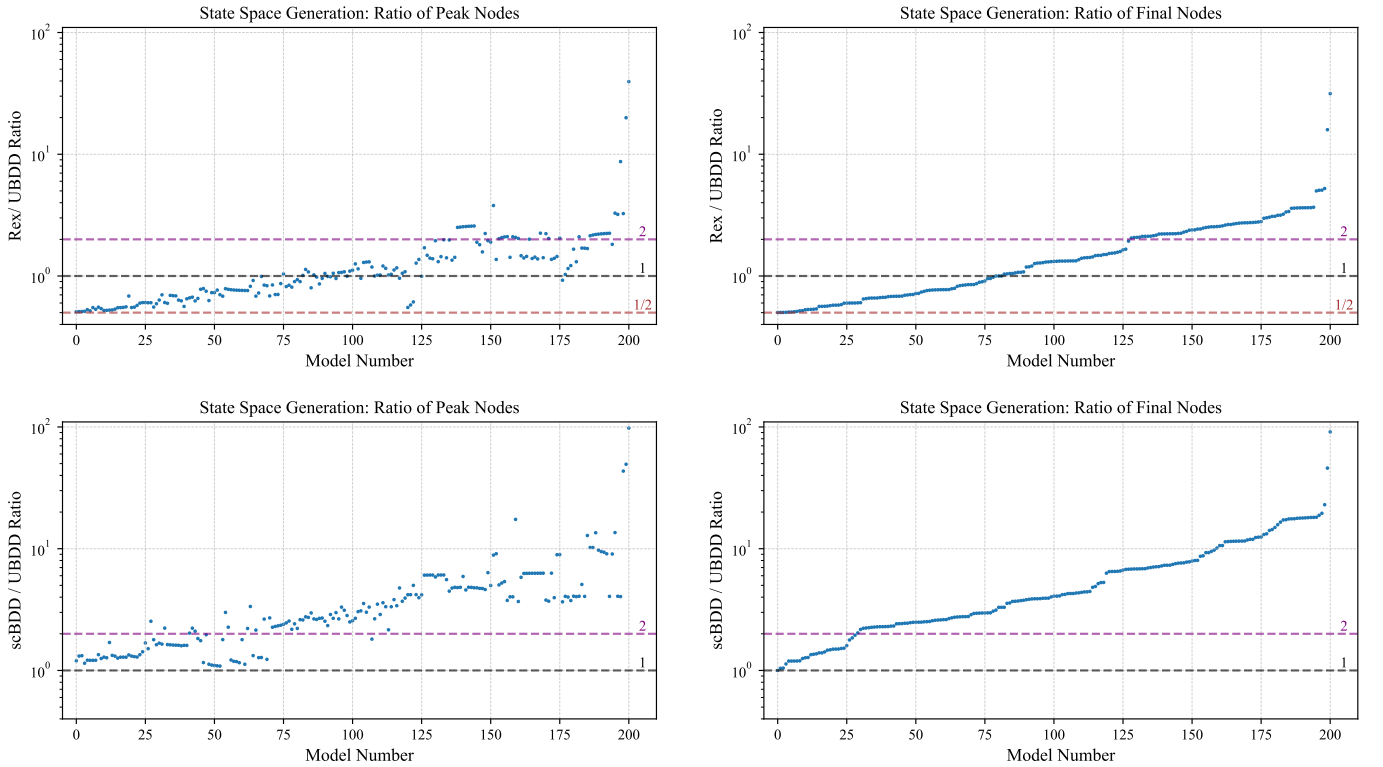


Fig. 10. Comparison of peak and final nodes for state space generation

to construct the result, in seconds. In addition to individual results, we also compute an aggregate score to evaluate overall performance,

$$s(v) = \sqrt[N]{\prod_{n=1}^N \alpha_v(n) / \min\{\alpha_{\text{scBDD}}(n), \alpha_{\text{Rex}}(n), \alpha_{\text{UBDD}}(n)\}}$$

where $s(v)$ is the “score” for variant v , calculated as the geometric mean [16] over N model instances, and $\alpha_v(n)$ is one of the measures (peak nodes, final nodes, or runtime) for variant v for model instance n . Intuitively, the score gives the “average” (as a geometric mean) factor increase a variant had

against the best-performing variant, with a score close to 1 meaning that the variant was usually the best.

A. Combinatorial Circuits

We tested all circuits from the LGSynth91 benchmark [17], and removed circuits that exceeded memory limits, or that required fewer than 10,000 peak nodes when using BDDs. We used the order of gates provided by each circuit to determine a fixed order for its L input variables, and used the same order for all BDD variants. Table III gives results for all 14 circuits, in increasing order of the ratio of RexBDD final nodes divided

TABLE IV
EXPERIMENTAL RESULTS FOR STATE SPACE GENERATION

Model	L	scBDD / UBDD			RexBDD / UBDD			UBDD		
		Peak	Final	Time	Peak	Final	Time	#Peak	#Final	Time (s)
shield_t_rv_050_a	403	1.086×	2.494×	0.993×	0.509×	0.501×	0.903×	7133955	403	407.05
MAPKbis-53-2-0	106	1.146×	1.131×	0.994×	0.529×	0.506×	0.945×	326415	267	42.11
des_02_a	123	1.287×	1.272×	1.005×	0.555×	0.518×	0.920×	5953299	247787	1037.55
shield_t_iip_010_a	193	1.125×	2.617×	0.987×	0.521×	0.528×	0.875×	37814014	193	1388.54
shield_t_ppp_005_a	128	1.234×	2.773×	1.007×	0.549×	0.563×	0.875×	12169464	128	478.34
dekker-15	75	1.286×	1.394×	0.997×	0.551×	0.576×	0.909×	4847756	270298	490.71
closed_system1	231	2.225×	2.318×	1.034×	0.574×	0.577×	0.973×	285547	7363	465.63
railroad-005-pt	68	2.026×	2.313×	1.005×	0.599×	0.582×	0.948×	28761	9861	15.32
RAS-C-100	600	1.211×	1.195×	1.005×	0.605×	0.598×	0.931×	2754833	3810	137.87
RAS-R-15	60	1.348×	1.200×	0.992×	0.556×	0.600×	0.924×	273855	163849	43.20
Sudoku-PT-A-N03	54	2.228×	2.233×	1.041×	0.592×	0.602×	0.953×	181276	73587	56.50
Peterson-2	102	2.101×	2.417×	1.066×	0.634×	0.604×	0.963×	211524	3480	131.94
rwmutex-r10w100	230	2.728×	3.584×	1.010×	0.700×	0.647×	0.985×	26033	14045	38.16
lambport_fmea-3	100	2.416×	3.131×	1.084×	0.605×	0.652×	0.963×	108781	11784	45.15
simple_lbs-5	59	1.509×	1.779×	1.010×	0.693×	0.659×	0.973×	153539	2780	91.98
Philosophers-10	50	1.627×	2.035×	1.027×	0.687×	0.660×	0.952×	666355	151688	178.01
shield_s_iip_002_a	53	1.286×	1.419×	1.008×	0.634×	0.665×	0.892×	2497911	1157	178.15
BART-005-unf	870	1.697×	1.347×	1.013×	0.625×	0.668×	0.966×	4923201	15663	1374.91
shield_s_ppp_002_a	65	1.285×	1.468×	1.007×	0.649×	0.677×	0.796×	16897156	2401	1928.40
shield_s_rv_004_a	59	1.299×	1.498×	1.024×	0.670×	0.681×	0.904×	3754056	865	196.79
bproc_03	274	3.353×	2.704×	1.103×	0.778×	0.684×	0.962×	592950	10725	257.07
anderson_05	161	2.702×	2.872×	1.087×	0.788×	0.695×	0.971×	1127886	45860	378.94
referendum-unfolded-15	46	1.330×	1.359×	1.006×	0.730×	0.704×	0.915×	6578078	434175	310.15
shared_memory-pt-10	131	2.536×	1.851×	1.076×	0.731×	0.718×	0.964×	105517	7948	38.02
gpufp_40_a	168	1.606×	2.296×	1.012×	0.759×	0.773×	0.905×	2402736	375	245.67
IBM703	262	3.008×	4.280×	1.052×	0.919×	0.791×	0.998×	127640	987	181.61
afcs_05_a	132	1.965×	2.444×	1.210×	0.683×	0.822×	0.957×	1739110	1114	523.12
distributeur-01-unfolded-02	24	1.289×	1.500×	0.986×	0.725×	0.833×	0.950×	33075	8190	4.01
medleyA13	173	4.998×	6.503×	1.199×	0.991×	0.840×	0.975×	294751	1313	145.31
resisol_09_1	299	5.573×	6.948×	1.008×	0.842×	0.844×	1.000×	298479	854	4790.05
eisenberg-mcguire_03	117	2.615×	3.301×	1.067×	0.704×	0.865×	0.954×	174482	4122	88.51
parking_2_8	137	2.158×	4.459×	1.076×	1.039×	0.911×	0.956×	189396	157	31.85
eratosthenes-500	499	1.201×	1.000×	1.020×	0.899×	1.000×	1.014×	3340873	95	1139.24
hrec_05	125	2.337×	3.842×	1.057×	0.955×	1.081×	0.977×	1022110	6260	504.12
raft_06	188	2.650×	3.702×	1.012×	1.306×	1.330×	0.920×	1526598	215	341.43
mainframe0-04-04	183	3.313×	3.916×	1.206×	1.033×	1.435×	0.956×	6355224	2379	1040.71
ARMCACHECoherence	87	2.503×	4.018×	1.007×	1.053×	1.500×	0.997×	397066	114	3031.86
stigelec11-a	156	4.190×	6.485×	0.985×	0.551×	1.532×	0.944×	70453	171	5145.59
firewire-16	254	5.224×	8.748×	1.231×	1.705×	1.660×	1.013×	913589	527	1268.48
dlcsh_4_a	1178	3.706×	11.937×	0.975×	1.401×	2.050×	1.017×	406222	238	4703.48
dlcfbar_4_a	927	3.752×	14.377×	0.968×	1.410×	2.119×	0.997×	177926	151	2792.06
discovery_15_a	153	6.085×	6.855×	1.054×	2.575×	2.226×	0.977×	12826	62	3.53
smhome_04	139	6.356×	7.799×	1.203×	2.234×	2.325×	1.004×	155926	573	61.20
autocar10_b	425	4.622×	7.764×	1.051×	2.034×	2.558×	0.982×	3350409	1366	3338.83
co4-11	545	10.230×	17.624×	1.245×	2.227×	2.736×	1.001×	510393	250	491.45
dlcro_13_a	463	4.044×	19.545×	1.022×	1.397×	2.764×	0.983×	50658	55	473.24
stigcomm09-a	364	9.740×	17.838×	1.008×	0.923×	2.981×	1.038×	53109	154	5321.52
champagne_H04_T1U	285	5.042×	8.666×	1.154×	2.099×	3.159×	0.929×	2954324	533	485.21
flexbar_22_a	267	6.295×	11.567×	1.063×	2.243×	3.642×	0.984×	80223	67	33.84
AirplaneLD-pt-0200	1419	97.816×	91.047×	1.049×	39.526×	31.475×	1.019×	76987	257	1471.07
Score		scBDD			RexBDD			UBDD		
N/A		3.694	5.386	1.120	1.322	1.574	1.002	1.193	1.172	1.049

by UBDD final nodes. The overall scores over all 14 circuits are given at the bottom of the table.

For scBDDs and RexBDDs, the table gives the factor increase as compared to UBDDs, with values greater than one indicating that UBDDs perform better, and values less than one indicating that UBDDs perform worse. For any particular circuit, we see that RexBDDs and UBDDs are competitive with each other, while scBDDs tended to perform considerably worse, although still within a factor of 2 most of the time. Looking at the overall scores, RexBDDs and UBDDs tend to be within a few percent of each other on average, with scBDDs

closer to 50 percent worse.

B. State Space Generation

We tested all safe Petri net models from the 2026 edition of the model checking contest [18], [19]. Most models have more than one *instance*, each obtained by fixing various model parameters. For each model instance, we used the saturation algorithm [20], modified slightly for each BDD variant, to generate the state space. For each instance, the ordering of its L variables is determined by the order of the places given in the Petri net input file. Again, we removed instances that

TABLE V
AIRPLANE_{LD} WITH VARYING PARAMETERS

Param.	L	REX / UBDD			UBDD		
		Peak	Final	Time	# Peak	# Final	Time (s)
20	159	3.793×	2.393×	1.00×	2,235	56	2.04
50	369	8.706×	5.071×	1.02×	4,995	56	14.92
100	719	19.945×	15.911×	1.03×	38,687	257	311.25
200	1419	39.526×	31.475×	1.05×	76,987	257	1598.65
500	3519	98.272×	78.167×	0.99×	191,887	257	13708.10
1000	7019	163.614×	89.893×	1.02×	92,395	56	27108.30

exceed memory limits, or require fewer than 10,000 peak ZDD nodes, leaving 201 model instances total.

Table IV reports detailed results for the largest instance of each model, in increasing order of (number of final RexBDD nodes) divided by (number of final UBDD nodes). The overall scores over all 201 model instances are given at the bottom of the table. Again, the scBDD and RexBDD columns give the factor increase as compared to UBDDs. Model instances near the top of the table are those for which RexBDDs outperform UBDDs, at most by a factor of $1/2$ (meaning, UBDDs are about $2\times$ the size of the RexBDD), as opposed to the factor of $1/3$ best-case behavior suggested in Sec. IV. Model instances near the middle of the table are those for which RexBDDs and UBDDs perform about the same, and model instances at the bottom are those for which UBDDs outperform RexBDDs. In particular, note that there are several instances where RexBDDs are more than a factor of $2\times$ the size of the UBDD. In contrast, for all but about a dozen of the instances, scBDDs are more than a factor of $2\times$ the size of the UBDD, for the final representation, with several instances worse than a factor of $10\times$ the UBDD size.

Note that the last model in Table IV, “Airplane_{LD}”, is an example where the RexBDD representation becomes increasingly larger than the UBDD representation, as the model parameter size increases. To emphasize this point, Table V shows results for this model with varying parameter sizes, running with increased memory limits to allow the study of larger instances. For this model, the UBDD final size remains fixed at either 56 or 257 nodes, depending on the order of places in the benchmark file. This indicates the presence of increased-length-and-chains or or-chains that UBDDs are able to exploit, but RexBDDs and the other variants are not.

Fig. 10 shows population plots for the increase in peak and final nodes, from UBDDs to RexBDDs (the top row of the figure) and UBDDs to scBDDs (the bottom row of the figure) for all 201 model instances. In each row, the instances are sorted from smallest to greatest factor for final node size. The dashed lines in the plots correspond to $1/2\times$ (UBDDs are twice as large as UBDDs), $1\times$, and $2\times$ (RexBDDs are twice as large as UBDDs). Considering the final size of RexBDDs, about 75 of the instances are better for RexBDDs, while the remaining instances are better for UBDDs. Instances from about 125 onward show UBDDs outperforming RexBDDs by more than a factor of 2. For scBDDs instead, instances from about 25 onward show UBDDs outperforming scBDDs by

more than a factor of 2, and instances from about 160 onward outperforming scBDDs by more than a factor of 10.

VI. CONCLUSION

We introduced UBDDs, a new variant of canonical BDDs that utilize three generalized reduction rules. Despite their conceptual simplicity, UBDDs are able to exploit all reduction patterns from several existing BDD variants, and some patterns that have not been exploited until now. As compared to other variants, UBDDs are at most a constant factor larger, but can be nearly a factor of L (the number of variables) smaller, for any function. Experimental results show that these performance differences can be realized in practice.

While UBDDs can dramatically reduce the number of nodes required as compared to RexBDDs and other variants, this currently does not translate to significant reduction in runtime. Improving the APPLY operation for UBDDs, so that more combinations of long edges can be exploited, is a direction for future work. Incorporating special edges for and-chains and or-chains to terminal nodes would allow UBDDs to represent RexBDD edges without requiring additional nodes. This would push the worst-case behavior of UBDDs closer to that of RexBDDs, at a cost of more complex reduction rules.

ACKNOWLEDGMENT

This work was supported in part by the National Science Foundation under award CCF-2212142.

DATA AVAILABILITY

The artifact(s) associated with this paper is/are available at Zenodo under the following DOI: 10.5281/zenodo.20144470.

REFERENCES

- [1] R. E. Bryant and Y.-A. Chen, “Verification of arithmetic circuits with binary moment diagrams,” in *Proc. 32nd ACM/IEEE Design Automation Conference*, 1995, pp. 535–541.
- [2] J. R. Burch, E. M. Clarke, D. E. Long, K. L. McMillan, and D. L. Dill, “Symbolic model checking for sequential circuit verification,” *IEEE Trans. CAD of Integr. Circ. and Syst.*, vol. 13, no. 4, pp. 401–424, Apr. 1994.
- [3] E. M. Clarke, O. Grumberg, H. Hiraishi, S. Jha, D. E. Long, K. L. McMillan, and L. A. Ness, “Verification of the Futurebus+ cache coherence protocol,” *Formal Methods in System Design*, vol. 6, no. 2, pp. 217–232, Mar 1995. [Online]. Available: <https://doi.org/10.1007/BF01383968>
- [4] M. Z. Kwiatkowska, G. Norman, and R. Segala, “Automated verification of a randomized distributed consensus protocol using cadence smv and prism,” in *Proc. CAV*. Springer, 2001, pp. 194–206.
- [5] J.-C. Madre and J.-P. Billon, “Proving circuit correctness using formal comparison between expected and extracted behaviour,” in *Proc. ACM/IEEE DAC*. IEEE CS Press, 1988, pp. 205–210.

- [6] S. Minato, N. Ishiura, and S. Yajima, “Shared binary decision diagram with attributed edges for efficient boolean function manipulation,” in *Proc. ACM/IEEE DAC*. IEEE CS Press, 1990, pp. 52–57.
- [7] R. E. Bryant, “Graph-based algorithms for boolean function manipulation,” *IEEE Trans. Comp.*, vol. 35, no. 8, pp. 677–691, Aug. 1986.
- [8] S. Minato, “Zero-suppressed BDDs and their applications,” *Software Tools for Technology Transfer*, vol. 3, pp. 156–170, 2001.
- [9] R. E. Bryant, “Chain reduction for binary and zero-suppressed decision diagrams,” *Journal of Automated Reasoning*, vol. 64, 10 2020.
- [10] T. van Dijk, R. Wille, and R. Meolic, “Tagged BDDs: combining reduction rules from different decision diagram types,” in *Proc. FMCAD*, 2017, pp. 108–115.
- [11] G. Ciardo, A. S. Miner, L. Deng, and J. Babar, “RexBDDs: reduction-on-edge complement-and-swap binary decision diagrams,” in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, ser. DAC ’24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: <https://doi.org/10.1145/3649329.3656533>
- [12] D. Miller and R. Drechsler, “Negation and duality in reduced ordered binary decision diagrams,” in *1997 IEEE Pacific Rim Conf. Communications, Computers and Signal Processing*, vol. 2, 1997, pp. 692–696.
- [13] J. Babar, G. Ciardo, and A. S. Miner, “CESRBDDs: binary decision diagrams with complemented edges and edge-specified reductions,” *Software Tools for Technology Transfer*, vol. 24, pp. 89–109, Feb. 2022.
- [14] J. Babar, C. Jiang, G. Ciardo, and A. Miner, “Binary decision diagrams with edge-specified reductions,” in *Proc. TACAS*. Springer, 2019, pp. 303–318.
- [15] “Bravedd.” [Online]. Available: https://github.com/lcdeng1/BraveDD_archive
- [16] P. J. Fleming and J. J. Wallace, “How not to lie with statistics: The correct way to summarize benchmark results,” *Commun. ACM*, vol. 29, no. 3, pp. 218–221, Mar. 1986. [Online]. Available: <http://doi.acm.org/10.1145/5666.5673>
- [17] S. Yang, *Logic synthesis and optimization benchmarks user guide: version 3.0*. Microelectronics Center of North Carolina (MCNC), 1991.
- [18] E. Amparore, B. Berthomieu, G. Ciardo, S. Dal Zilio, F. Gallà, L. M. Hillah, F. Hulin-Hubard, P. G. Jensen, L. Jezequel, F. Kordon, D. Le Botlan, T. Liebke, J. Meijer, A. Miner, E. Paviot-Adet, J. Srba, Y. Thierry-Mieg, T. van Dijk, and K. Wolf, “Presentation of the 9th edition of the model checking contest,” in *Proc. TACAS*, ser. LNCS 11429, D. Beyer, M. Huisman, F. Kordon, and B. Steffen, Eds. Prague, Czech Republic: Springer, Apr. 2019, pp. 50–68.
- [19] “Model checking contest 2026.” [Online]. Available: <https://mcc.lip6.fr/2026/>
- [20] G. Ciardo, G. Lüttgen, and R. Siminiceanu, “Saturation: An efficient iteration strategy for symbolic state space generation,” in *Proc. TACAS*, ser. LNCS 2031. Springer, 2001, pp. 328–342.